

Introduction to Asymptotic Performance Analysis of Algorithms

1. Motivation

When we design algorithms, we care about how efficiently they run as the input size grows. Instead of measuring actual execution time (which depends on hardware, compiler, and implementation details), we use mathematical tools to describe the “growth rate” of an algorithm’s runtime. This is called **asymptotic analysis**.

2. Growth of Functions

Suppose the runtime of an algorithm is expressed as a function $T(n)$, where n is the input size. For example:

- A linear search through n items might take $T(n) = cn$ steps.
- A binary search might take $T(n) = c \log_2 n$ steps.
- A nested loop over all pairs might take $T(n) = cn^2$ steps.

We ignore constant factors (c) and low-order terms, because asymptotically they do not affect the growth trend.

3. Big-O Notation

Definition: A function $T(n)$ is in $O(f(n))$ if there exist positive constants c and n_0 such that

$$T(n) \leq c \cdot f(n) \quad \text{for all } n \geq n_0.$$

Intuition: Big-O gives an *upper bound* on the runtime growth.

Examples:

- $3n + 5 \in O(n)$.
- $7n^2 + 2n + 8 \in O(n^2)$.
- $100n \log n + 50n \in O(n \log n)$.

4. Theta Notation

Definition: A function $T(n)$ is in $\Theta(f(n))$ if there exist positive constants c_1, c_2 , and n_0 such that

$$c_1 f(n) \leq T(n) \leq c_2 f(n) \quad \text{for all } n \geq n_0.$$

Intuition: $\Theta(f(n))$ means $f(n)$ is both an upper and lower bound for $T(n)$, so it captures the algorithm’s growth rate exactly (within constant factors).

Examples:

- $5n + 3 \in \Theta(n)$.
- $4n^2 + 2n + 7 \in \Theta(n^2)$.
- $10n \log n - 50n \in \Theta(n \log n)$.

5. Common Growth Rates

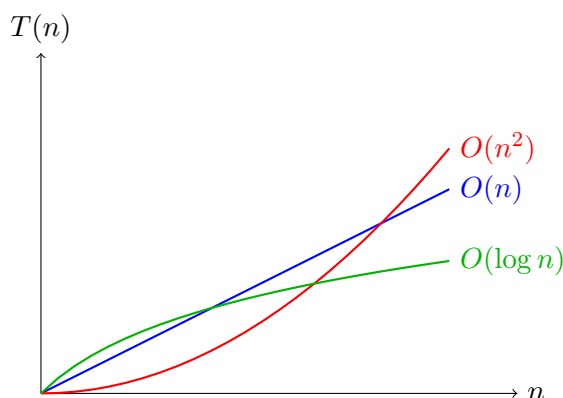
Function	Name
$O(1)$	Constant time
$O(\log n)$	Logarithmic time (e.g., binary search)
$O(n)$	Linear time (e.g., linear search)
$O(n \log n)$	Linearithmic time (e.g., merge sort, quicksort average case)
$O(n^2)$	Quadratic time (e.g., bubble sort, insertion sort worst case)
$O(n^3)$	Cubic time (e.g., some dynamic programming)
$O(2^n)$	Exponential time (e.g., brute-force subset search)
$O(n!)$	Factorial time (e.g., traveling salesman brute force)

6. Example: Comparing Algorithms

- Algorithm A: $T_A(n) = 100n$.
- Algorithm B: $T_B(n) = n^2$.

For small n , $T_B(n)$ may be smaller. But as n grows, n^2 dominates $100n$. Thus, Algorithm A ($O(n)$) is asymptotically faster than Algorithm B ($O(n^2)$).

7. Visualizing Growth



8. Key Takeaways

- Asymptotic analysis abstracts away machine-dependent details.
- Big-O notation: upper bound (worst-case growth).
- Theta notation: tight bound (exact growth within constants).
- Constant factors and lower-order terms are ignored.
- Choosing an algorithm with better asymptotic complexity matters most for large inputs.

Further Reading

- T. Cormen, C. Leiserson, R. Rivest, and C. Stein. *Introduction to Algorithms*. MIT Press.
- J. Kleinberg and É. Tardos. *Algorithm Design*. Pearson.